

Descrição do Projeto: Sistema de Atendimento

Visão Geral

O **Sistema de Atendimento** é uma solução digital robusta e intuitiva projetada para otimizar o fluxo de comunicação entre organizações e seus clientes. O objetivo central é centralizar a gestão de chamados, permitindo que solicitações de suporte sejam registradas, acompanhadas e resolvidas de forma eficiente e transparente. Através de uma interface moderna e fluxos de trabalho automatizados, o sistema reduz o tempo de resposta e eleva a satisfação do usuário final, enquanto fornece ferramentas de controle e auditoria para a gestão.

Escopo e Objetivos

O sistema visa atender às necessidades de três perfis distintos, garantindo que cada um tenha as ferramentas necessárias para cumprir seu papel:

- Agilidade para o Cliente:** Autonomia para cadastro e acompanhamento de tickets.
- Eficiência para o Atendente:** Organização centralizada para triagem e resposta de demandas.
- Controle para o Administrador:** Supervisão total da equipe, auditoria de ações e análise de desempenho através de relatórios.

Arquitetura do Sistema

A solução é fundamentada no padrão de arquitetura **MVC (Model-View-Controller)**, garantindo uma estrutura modular, escalável e de fácil manutenção. Esta abordagem permite que a lógica de negócio, a interface do usuário e o controle de dados operem de forma independente, facilitando atualizações futuras e a integração de novas funcionalidades sem comprometer a estabilidade do núcleo do sistema.

Plataforma Tecnológica

A escolha das tecnologias prioriza a estabilidade, segurança e rapidez na entrega:

- Linguagem de Programação:** Python 3.10+ (Conhecida pela legibilidade e vasto ecossistema).
- Framework Web:** Flask (Abordagem minimalista e eficiente).
- Motor de Templates:** Jinja 2 (Para renderização dinâmica de conteúdo).
- Interface (Front-end):** HTML5, CSS3 e Bootstrap 5 (Design responsivo e acessível).

- **Banco de Dados:** SQLite (Garante portabilidade e simplicidade na gestão de dados).
- **Infraestrutura:** Docker (Ambientes isolados e replicáveis para desenvolvimento e produção).
- **Versionamento:** Git/GitHub (Fluxo de trabalho profissional e seguro).

Estrutura de Diretórios

O projeto segue uma organização lógica e padronizada, facilitando a navegação de desenvolvedores:

```
sistema-atendimento/  
├── app/                    # Núcleo da aplicação Flask  
│   ├── controllers/       # Controladores e Blueprints (Lógica de rotas)  
│   ├── models/            # Modelos de Dados (ORM SQLAlchemy)  
│   ├── templates/         # Visualização (Templates HTML/Jinja2)  
│   ├── database.py         # Configuração da conexão com o banco  
│   └── __init__.py         # Inicialização e Factory da aplicação  
├── doc/                   # Documentação e especificações de design  
├── tests/                 # Suite de testes para garantia de qualidade  
├── .env.example           # Template de configuração de ambiente  
├── config.py              # Gerenciamento de configurações globais  
├── Dockerfile             # Receita de construção do container  
├── requirements.txt        # Listagem de dependências Python  
├── run.py                 # Script de entrada para execução do sistema  
└── especificacao.md        # Documentação descritiva do projeto
```

Convenções de Desenvolvimento

Para manter a integridade e qualidade do código, adotamos as seguintes convenções:

- **PEP 8:** Padrão oficial de estilo para código Python.
- **Snake_Case:** Utilizado para nomes de variáveis, funções e arquivos Python.
- **Mensagens de Commit:** Padronizadas e em português, descrevendo claramente o propósito da alteração.
- **Segurança:** Uso obrigatório de hashing para senhas e variáveis de ambiente para dados sensíveis.

Serviços e Funcionalidades Principais

- **Módulo de Autenticação:** Gestão segura de sessões para todos os tipos de usuários.
- **Painel de Solicitações:** Sistema completo de abertura, consulta e histórico de tickets.
- **Auditoria de Operações:** Sistema de logs para rastreamento de ações críticas.
- **Gestão de Equipe:** Interface administrativa para controle de usuários atendentes.
- **Módulo de Relatórios:** Extração de métricas gerenciais sobre a eficiência do atendimento.

Variáveis de Ambiente

O sistema exige as seguintes chaves de configuração no arquivo `.env` :

- `SECRET_KEY` : Proteção de sessões e cookies.
- `DATABASE_URI` : Localização e credenciais do banco de dados.
- `DEBUG_MODE` : Controle do modo de desenvolvimento.
- `PROPRIETARIO_EMAIL` e `PROPRIETARIO_PASSWORD` : Credenciais para o administrador mestre do sistema.

Perfis de Usuário e Acessos

- **Administrador:** Perfil de gestão. Realiza o cadastro de atendentes, visualiza logs de auditoria e analisa relatórios de desempenho. O sistema conta com um "Administrador Inicial" configurado via variáveis de ambiente.
- **Atendente:** Perfil operacional. Recebe as demandas dos clientes, interage através de respostas e gerencia o status dos atendimentos até a resolução.
- **Cliente:** Perfil solicitante. Possui capacidade de autocadastro e é o responsável por iniciar o fluxo de atendimento através da criação de solicitações